MAREVLO · PRODUCTION RAG

Better Retrieval: Hybrid Search & Token Budgeting

Fixing two quiet weaknesses in basic retrieval — the exact-term blind spot, and the hidden cost of stuffing in context.

PART II — MAKING IT WORK

Module 7

Production RAG: Building Retrieval-Augmented Generation Systems That Survive Real Users

What this module is about

In Module 5 we built a working RAG system that fetches "the nearest chunks" by meaning and answers from them. That system is sound, but it carries two quiet weaknesses that surface the moment real users start asking real questions. The first is a blind spot in how it searches: searching purely by meaning misses exact terms like product codes, model numbers, and names. The second is a hidden cost in how it assembles context: simply grabbing a fixed number of chunks ignores how much room they actually take up, which wastes money, slows answers, and can even make them worse. This module fixes both.

The two topics are separate, so the module is in two halves. Part A is **hybrid search** — combining meaning-based search with old-fashioned keyword search so that nothing slips through. Part B is **token budgeting** — being deliberate about how much context we send, measured the way the model actually counts it. Both are the kind of improvement that does not show up in a quick demo but makes an enormous difference once the system meets the messy variety of real questions.

> **THE WHOLE MODULE IN ONE SENTENCE**
> Search by meaning *and* by exact words so you never miss a code or a name, then send a deliberate, token-measured amount of context rather than a careless pile of chunks.

Where we are

We are improving the query pipeline we built in Module 5. The retrieval step gets smarter (hybrid search) and the prompt-building step gets disciplined about size (token budgeting). Nothing about the indexing pipeline changes; this is all about how we search and assemble at query time.

PART A

Hybrid search: meaning and exact words together

SECTION 1

The blind spot of searching by meaning

Semantic search is wonderful at meaning and surprisingly bad at exact terms. Here is exactly where it fails.

Everything we built in Modules 3 and 4 searches by **meaning**. We turn text into embeddings — points on a meaning map — and find the nearest ones. This is genuinely powerful: it matches a question to an answer even when they share no words, because it understands that "how do I clean it?" and "wipe the parts after each use" mean the same thing. For most questions, this is exactly what you want.

But meaning-based search has a specific, predictable weakness, and it is worth understanding precisely *why*, not just that it exists. An embedding captures meaning, and some pieces of text carry almost no meaning to capture. Consider an error code like "E-07," a model number like "X1-Pro," or an invoice number like "88213." These are essentially labels — they do not *mean* anything in the way a sentence does, so the embedding model has very little to work

with. The result is that the embedding of "E-07" floats around in a vague region of the meaning map, close to nothing in particular, and a search for it returns chunks about errors in general while completely missing the one chunk that actually contains "E-07."

THINK ABOUT IT

Suppose our coffee-machine manual has one line: "Error E-07 means the water tank is empty." A user types "what is error E-07?" Meaning-based search embeds "error E-07," but "E-07" carries almost no meaning, so the query drifts toward generic "error" chunks. The one line that actually answers it may not even make the top results — even though, to a human, it is an obvious exact-word match. That gap is the blind spot we are about to close.

SECTION 2

The other half: keyword search

The oldest kind of search is exactly strong where meaning-based search is weak. The two are natural partners.

Long before embeddings, search worked by matching **words**. If you searched for "E-07," it found the documents containing the exact characters "E-07." This is keyword search, and the standard modern version of it is a method called BM25, which you can think of simply as a smart way of scoring how well a document's words match the words in your query — rewarding rare, distinctive words and not being fooled by very common ones.

Keyword search is the mirror image of meaning-based search. It is excellent at exactly what embeddings are bad at: exact terms, codes, names, and rare jargon, because it matches the literal characters. And it is bad at exactly what embeddings are good at: it has no notion of meaning, so a search for "car" will not find a document that only says "automobile," and a paraphrased question will miss an answer worded differently. Neither method is complete on its own, but their weaknesses are opposite — which is precisely what makes combining them so effective.

	Meaning-based search (embeddings)	Keyword search (BM25)
Great at	Paraphrases, synonyms, the general sense of a question	Exact terms: codes, model numbers, names, rare jargon
Bad at	Exact strings that carry little meaning ("E-07")	Anything worded differently from the document ("car" vs "automobile")
Matches on	Nearness on the meaning map	The literal words that appear

SECTION 3

Combining them: hybrid search

Run both searches, then merge their results so a chunk that wins either way rises to the top.

Hybrid search is exactly what it sounds like: run both searches — meaning-based and keyword — and combine their results into one ranked list. A chunk that scores well in *either* method should appear near the top, and a chunk that scores well in *both* should appear at the very top. That way, a paraphrased question is caught by the meaning side, an exact code is caught by the keyword side, and nothing relevant slips through.

The interesting question is *how* to combine them, because the two methods produce scores on completely different scales that cannot be compared directly. A meaning-based similarity of "0.82" and a keyword score of "14.3" are measured in different units; adding them together is meaningless. The clean solution is to ignore the raw scores and use only the **ranks** — the positions each chunk took in each list. This method is called Reciprocal Rank Fusion, and the idea is simple: each chunk earns points based on how high it ranked in each list, and we add up the points. A chunk ranked first earns the most points, second a little less, and so on; summing across both lists rewards chunks that ranked well in either or both.

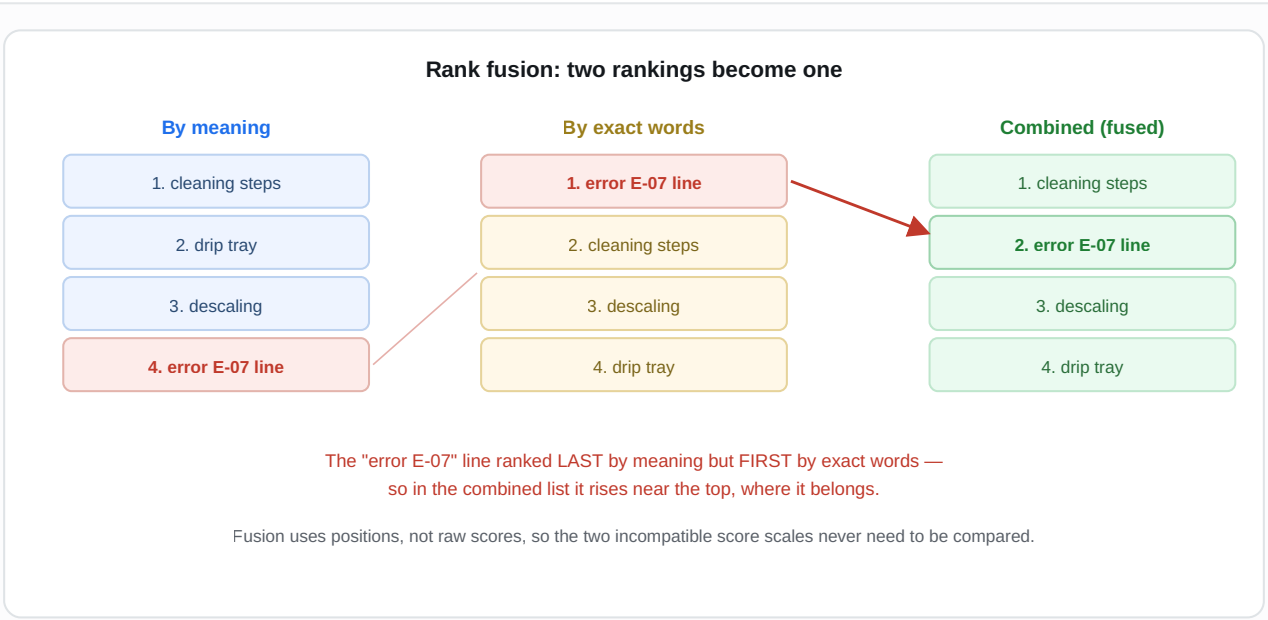

Figure 1. Hybrid search runs both methods and fuses their rankings. The exact-term chunk that meaning-based search nearly missed is rescued by the keyword side and lands near the top of the combined list.

ANALOGY — ASKING TWO DIFFERENT EXPERTS

Imagine you have two assistants. One deeply understands what you *mean* but is hazy on exact names and numbers. The other is a meticulous index-keeper who finds exact words instantly but has no sense of meaning. If you ask only one, you get half a service. If you ask both and combine their recommendations — trusting whatever either one ranks highly — you get answers that are both meaningful and exact. Hybrid search is simply consulting both experts and merging their advice.

Below is our own implementation. We write the fusion step by hand so you can see there is nothing mysterious in it: it simply gives each chunk points based on its rank in each list and adds them up.

LAB 7.1 — Hybrid search with rank fusion written by hand (our own)

```
# A keyword retriever (BM25) over our chunks. Verify the import against
# your installed version.
from langchain_community.retrievers import BM25Retriever


def reciprocal_rank_fusion(list_of_rankings, k=60):
    """Merge several ranked lists into one, using positions not raw scores.

    Each chunk earns 1 / (k + rank) points from every list it appears in,
    where rank starts at 1 for the top item. We add up the points. The
    constant k just softens the gap between ranks; 60 is a common choice.
    """
    points = {}
    for ranking in list_of_rankings:
        for rank, chunk_text in enumerate(ranking, start=1):
            points[chunk_text] = points.get(chunk_text, 0) + 1 / (k + rank)
    # Sort chunks by total points, best first.
    return sorted(points, key=points.get, reverse=True)


def hybrid_search(question, all_chunks, k=4):
    """Run meaning-based AND keyword search, then fuse the two rankings."""
    # 1) Meaning-based ranking (our store from Module 4).
    semantic_hits = store.similarity_search(question, k=10)
    semantic_ranking = [doc.page_content for doc in semantic_hits]

    # 2) Keyword ranking (BM25 over the same chunks).
    keyword = BM25Retriever.from_texts(all_chunks)
    keyword.k = 10
    keyword_ranking = [doc.page_content for doc in keyword.invoke(question)]

    # 3) Fuse the two rankings and keep the top k.
    fused = reciprocal_rank_fusion([semantic_ranking, keyword_ranking])
    return fused[:k]
```

WHEN HYBRID SEARCH IS WORTH IT — AND WHEN IT IS NOT

Hybrid search shines when your documents are full of exact terms — error codes, part numbers, names, technical jargon. If your content is flowing prose with few such terms, the keyword half may add little, and it does cost extra work on every query. As always in this course: turn it on when your documents and your failing questions justify it, and measure the difference rather than assuming. Sophistication is a cost, not a free upgrade. **MEDIUM CONFIDENCE — DEPENDS ON YOUR CONTENT**

PART B

Token budgeting: sending the right amount of context

SECTION 4

Why more context is not free

Every chunk you add to the prompt costs you in three ways. "Just retrieve more" is one of the most expensive habits in RAG.

It is tempting to think that giving the model more context can only help — if four chunks are good, surely twelve are better. This instinct is wrong, and understanding exactly why protects you from three real costs that compound on every single question your system answers.

The first cost is **money**. Language models charge by the token — roughly, by the amount of text they read and write. Every extra chunk you include is more tokens on every query, and at thousands of queries a day, careless context is a line item that can quietly dominate your bill. The second cost is **speed**. More text takes the model longer to read and respond to, so a bloated prompt makes every answer slower, which users feel directly. The third cost is the most surprising: **quality can actually fall**. Models pay the most attention to the beginning and end of a long prompt and can overlook information buried in the middle, so a genuinely relevant chunk drowned among a dozen mediocre ones may be effectively ignored. Past a point, adding context does not add answers — it adds noise.

> **THE KEY SHIFT IN THINKING**
>
> Stop measuring context by the *number* of chunks and start measuring it by the number of *tokens*. A chunk count is a poor guide because chunks vary wildly in size — four chunks might be 400 tokens or 4,000. What the model actually cares about, and what you actually pay for, is tokens. So we set a token budget and fill it deliberately.

SECTION 5

Budgeting in tokens, not chunk counts

Decide how many tokens of context you will allow, then add chunks by relevance until the budget is full — and stop.

The fix is to choose a token ceiling for your retrieved context — say, 1,500 tokens — and then add chunks one at a time, best first, until adding the next one would exceed the ceiling. At that point you stop. This way the amount of context is controlled regardless of how big the individual chunks happen to be, and you never accidentally send a wall of text just because a few chunks were unusually long. Crucially, you must also leave **headroom**: the model's total limit has to hold not just your context, but also your instructions, the user's question, and the answer the model still has to write. Spend your whole limit on context and there is no room left for the reply.

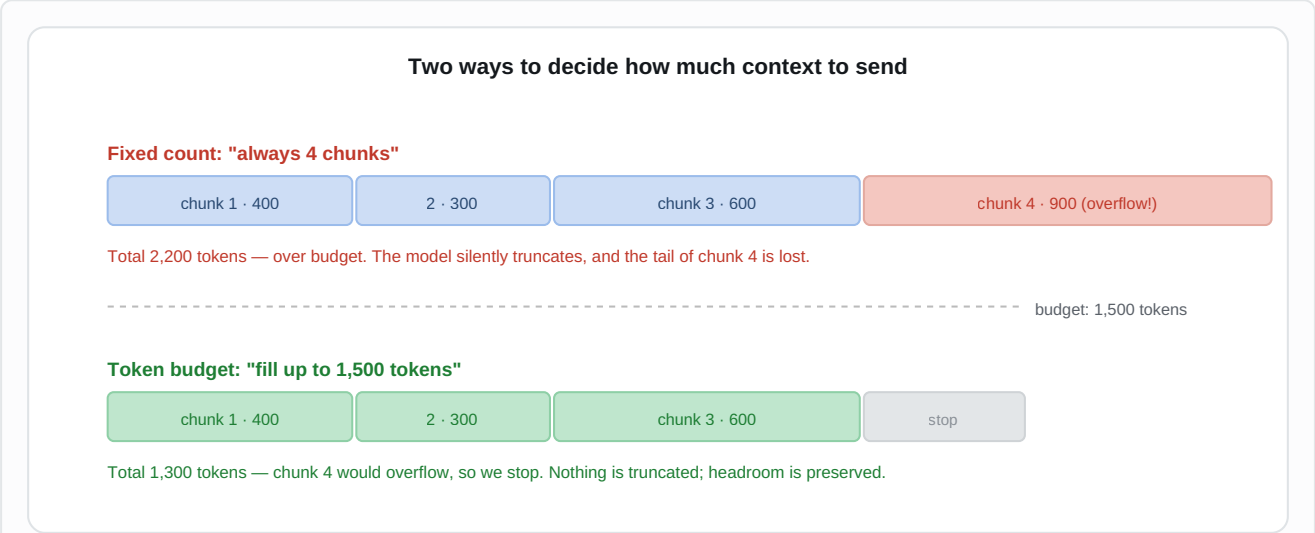

Figure 2. A fixed chunk count ignores size and can overflow the budget, causing silent truncation. A token budget adds chunks by relevance until the next would overflow, then stops — keeping context controlled and leaving headroom for the answer.

Here is our own helper that does this. Notice it measures length in tokens using a counter that matches the model — counting characters instead would be the same mistake we warned about back in Module 2, because dense text packs many tokens into few characters.

LAB 7.2 — Fit chunks to a token budget (our own helper)

```
import tiktoken

encoder = tiktoken.get_encoding("cl100k_base")   # match this to your model

def count_tokens(text):
    """How many tokens this text uses, the way the model counts."""
    return len(encoder.encode(text))


def fit_to_budget(chunks, max_tokens=1500):
    """Add chunks best-first until the next one would exceed the budget.

    `chunks` should already be ordered best-first (e.g. from hybrid_search).
    We stop as soon as adding another chunk would overflow, so the context
    stays within budget no matter how big individual chunks are.
    """
    kept = []
    used = 0
    for chunk in chunks:
        needed = count_tokens(chunk)
        if used + needed > max_tokens:   # next chunk would overflow -> stop
            break
        kept.append(chunk)
        used += needed
    return kept
```

With hybrid search and token budgeting in place, our retrieval step is now both thorough and disciplined: it catches meaning and exact terms alike, and it sends a controlled, well-sized amount of context every time. The two

improvements work together — hybrid search finds the best chunks, and the token budget keeps the best few rather than an unwieldy pile.

SECTION 6

Faults and corrections

The mistakes people make when adding hybrid search and managing context. For each: the broken version, why it breaks, and the fix.

FAULT 1 — Relying on meaning-based search alone for exact terms

THE BROKEN APPROACH

```
# Users search for codes and part numbers, but we only search by meaning.
results = store.similarity_search("error E-07", k=4)   # may miss the E-07 line
```

WHY IT BREAKS

Exact terms like "E-07" carry almost no meaning for an embedding to capture, so meaning-based search drifts toward generic chunks and can miss the one chunk that literally contains the term. The user gets a vague non-answer to a precise question, and it feels like the information is missing when it is sitting right there in the documents.

THE CORRECTION

```
# Add keyword search alongside meaning-based search and fuse them.
results = hybrid_search("error E-07", all_chunks, k=4)   # keyword side catches it
```

FAULT 2 — Adding the two search scores together directly

THE BROKEN APPROACH

```
# The two scores are on totally different scales, so adding them is
# meaningless -- whichever scale is bigger silently dominates.
combined = semantic_score + keyword_score   # e.g. 0.82 + 14.3 -> nonsense
```

WHY IT BREAKS

A meaning-based similarity might range from 0 to 1, while a keyword score might range into the tens. Adding them lets the larger-scaled score completely overwhelm the smaller one, so one method effectively gets ignored and your "hybrid" search is hybrid in name only. The two numbers are in different units and were never meant to be added.

THE CORRECTION

```
# Combine by RANK, not raw score, using rank fusion. Positions are
# comparable across methods even when the scores are not.
fused = reciprocal_rank_fusion([semantic_ranking, keyword_ranking])
```

FAULT 3 — Turning on hybrid search everywhere without measuring

THE BROKEN ASSUMPTION

```
# "Hybrid is more advanced, so use it for everything" -- even for a corpus
# of flowing prose with almost no codes or names. Extra work, little gain.
results = hybrid_search(question, all_chunks)   # on every query, always
```

WHY IT BREAKS

Hybrid search costs extra work on every query — you run a second search and a fusion step. That cost is well repaid when your documents are full of exact terms, but on pure prose with few codes or names, the keyword half adds little and you pay the cost for almost no benefit. Adding sophistication you have not shown you need is itself a kind of mistake.

THE CORRECTION

```
# Measure both on a fixed set of real questions, then choose. Keep hybrid
# only if it actually improves YOUR results.
# Compare meaning-only vs hybrid on your own eval questions before committing.
```

FAULT 4 — Using a fixed chunk count as the budget

THE BROKEN APPROACH

```
# "Always 8 chunks" ignores that chunks vary in size. Eight big chunks
# can blow past the model's limit, causing silent truncation.
chunks = retrieve(question, k=8)
prompt = build_prompt(question, chunks)   # might overflow without warning
```

WHY IT BREAKS

A chunk count is a poor proxy for size. Eight short chunks might fit comfortably while eight long ones overflow the model's limit, at which point the prompt is silently cut off and part of your context — or even the question — vanishes. Because nothing errors, the answer just quietly gets worse, and the cause is invisible unless you measure tokens.

THE CORRECTION

```
# Budget by tokens, so context is controlled regardless of chunk size.
chunks = retrieve(question, k=12)            # retrieve a generous pool...
chunks = fit_to_budget(chunks, max_tokens=1500)   # ...then trim to fit
```

FAULT 5 — Leaving no headroom for the question and the answer

THE BROKEN APPROACH

```
# Spending the model's entire limit on retrieved context, leaving no
# room for the instructions, the question, or the answer it must write.
chunks = fit_to_budget(chunks, max_tokens=model_limit)   # uses 100% of the limit
```

WHY IT BREAKS

The model's total limit must hold everything: your instructions, the retrieved context, the user's question, and the answer the model still has to generate. If context eats the entire budget, there is no space left for the reply, so the answer gets cut short or the request fails outright. The budget for context must be only a portion of the total, with the rest reserved.

THE CORRECTION

```
# Reserve room for instructions, question, and answer. Give context only
# a share of the total limit.
reserved = 1000   # for instructions + question + the answer
context_budget = model_limit - reserved
chunks = fit_to_budget(chunks, max_tokens=context_budget)
```

Checkpoint: try it yourself

This exercise lets you feel both improvements directly — first the exact-term blind spot and its cure, then the difference between counting chunks and counting tokens. It takes about thirty minutes.

1. Add one chunk to your store that contains a made-up exact term, for example "Error E-07 means the water tank is empty." Then search for "what is error E-07?" using meaning-based search alone, and note where (or whether) that chunk appears in the results.
2. Now run the same query through `hybrid_search` from Lab 7.1 and compare. The keyword side should pull the exact-term chunk up near the top. You have just watched hybrid search close the blind spot.
3. Take a handful of chunks of very different lengths and count the tokens of each with `count_tokens`. Notice how little the chunk *count* tells you about the total size.
4. Run `fit_to_budget` on those chunks with a small budget and watch it stop partway through, keeping only what fits. Compare that to simply taking a fixed number of chunks, and confirm for yourself why the token budget is the safer rule.

> **WHAT YOU SHOULD NOTICE**
>
> In step 2, the retrieval did not get "smarter" in some vague way — a second, very literal kind of search rescued a chunk the first kind nearly missed. And in step 4, you saw that "four chunks" and "a controlled amount of context" are not the same thing at all. Both lessons share a theme: precise control beats hopeful approximation.

What to carry forward

> **THE SIX THINGS TO REMEMBER**
>
> **One,** meaning-based search has a blind spot for exact terms — codes, model numbers, names — because they carry little meaning to embed. **Two,** keyword search is strong exactly where meaning-based search is weak, so the two are natural partners. **Three,** hybrid search runs both and fuses their rankings, so a chunk that wins either way rises to the top. **Four,** fuse by rank, never by adding raw scores, because the two scales are not comparable. **Five,** measure context in tokens, not chunk counts, because more context costs money, speed, and sometimes quality. **Six,** set a token budget and reserve headroom for the instructions, question, and answer — fill the budget deliberately rather than grabbing a careless pile of chunks.

Our retrieval is now thorough and disciplined — but we still cannot see inside it. When an answer is wrong, or slow, or expensive, how do we look at exactly what happened on that one request, and how do we track the system's health over time? That is observability, and it is the subject of the next module. You cannot improve what you cannot see, and so far our system has been a black box.